

UCS645

PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 7

Submitted by: Ritwik Gupta (102303096)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Problem 1: Sum of First N Integers (Multi-Task Kernel)

1. Objective

To implement a CUDA kernel where different threads perform entirely different tasks to compute the sum of the first N integers, comparing an iterative approach and a direct mathematical formula approach.

2. Problem Description

Two distinct methods are used to compute the sum of the first N integers ($N = 1024$):

- **Iterative Approach:** Sum computed using a standard loop.
- **Formula Approach:** Using the mathematical formula $n(n+1)/2$. Each method is mapped to a separate thread within the CUDA kernel.

3. Methodology

1. Defined $N=1024$.
2. Allocated an output array of size 2 on the device (Index 0 for iterative, Index 1 for formula).
3. Launched a single block with 2 threads.
4. Thread 0 executes the `for` loop to compute the sum.
5. Thread 1 computes the sum via $n(n+1)/2$.
6. Copied the results back to the host for verification.

4. Results

- **Iterative Sum:** 524800
- **Formula Sum:** 524800
- Both threads produced identical and correct results simultaneously.

5. Discussion

- **Formula-based computation** is extremely fast with $O(1)$ complexity.
- **Iterative approach** incurs a delay scaling with $O(N)$ due to loop operations.
- While thread divergence occurs (since threads in the same warp execute completely different instructions), the computation is small enough that the GPU easily handles it. This demonstrates the GPU's ability to handle concurrent task parallelism, not just data parallelism.

6. Conclusion

CUDA successfully executes multiple kernels/threads performing entirely different logical tasks. The formula-based approach is vastly more efficient, highlighting the importance of algorithm selection even on highly parallel hardware.

Problem 2: Parallel Merge Sort

1. Objective

To implement merge sort using parallel techniques and compare CPU execution with CUDA-based parallel execution.

2. Problem Description

An array of size 1000 must be sorted using two approaches:

- **Sequential/Pipelined Approach:** A standard CPU-based merge sort.
- **CUDA Parallel Sorting:** A bottom-up parallel merge sort executed on the GPU.

3. Methodology

1. Generated a random array of 1000 integers.
2. Implemented standard recursive CPU merge sort to act as the baseline.
3. Implemented a bottom-up CUDA merge sort, where each thread merges adjacent sub-arrays in parallel. The width of the sub-arrays doubles with each kernel launch.
4. Recorded and compared execution times using timing functions.

4. Results

- **CPU Sort Time:** 0.501 ms
- **CUDA Sort Time:** 1.003 ms
- Sorted output verified for correctness against the CPU version.

5. Discussion

- For a highly small dataset ($N=1000$), the CPU outperforms the GPU (0.5 ms vs 1.0 ms). This perfectly demonstrates **kernel launch overhead**. The time it takes to allocate device memory, transfer the data via PCIe, and schedule the thread blocks on the GPU outweighs the actual computation time required to sort 1,000 integers.
- CUDA exploits vast data parallelism by merging independent pairs of sub-arrays simultaneously. As N scales to millions, this parallel capability easily overtakes the sequential limitations of the CPU.

6. Conclusion

Parallel sorting using CUDA successfully executes, but the performance metrics highlight a critical concept in heterogeneous computing: GPUs are highly inefficient for small workloads due to communication and synchronization overhead. Algorithm selection must account for dataset size.

Problem 3: Vector Addition + Profiling

1. Objective

To implement a vector addition kernel utilizing statically defined global variables (`__device__` memory) and analyze its performance using memory bandwidth calculations and timing events.

2. Problem Description

- Perform vector addition: $C[i] = A[i] + B[i]$.
- Memory is declared statically at compile-time on the device rather than using `cudaMalloc`.
- Calculate and compare **Theoretical Memory Bandwidth** and **Measured Memory Bandwidth**.

3. Methodology

1. Declared static global arrays using `__device__`.
2. Copied data to the static arrays using `cudaMemcpyToSymbol`.
3. Recorded execution time using `cudaEvent_t`.
4. Queried device properties using `cudaGetDeviceProperties` to extract the memory clock rate and bus width.
5. Calculated metrics based on the provided formulas.

4. Results

- **Kernel Execution Time:** 0.369056 ms
- **Theoretical Bandwidth:** 40.080000 GB/s
- **Measured Bandwidth:** 32.515392 GB/s

5. Analysis

- **Theoretical Bandwidth:** $BW = 2 \times \text{memoryClockRate} \times \text{memoryBusWidth}$
- **Measured Bandwidth:**
 - $BW = \text{Time (seconds)} \times (\text{Reads (Bytes)} + \text{Writes (Bytes)})$
 - Where Reads = $2N \times 4$ bytes (floats), and Writes = $N \times 4$ bytes.

6. Discussion

The kernel achieved roughly **81%** of the theoretical maximum bandwidth (32.51/40.08). This is a highly efficient result. The gap between measured and theoretical bandwidth is accounted for by standard memory latency, unavoidable kernel overhead, and the limits of the memory controller.

7. Conclusion

CUDA vector addition via statically allocated memory was executed successfully. The bandwidth profiling effectively demonstrates that while theoretical limits dictate absolute maximums, real-world measured bandwidth is slightly lower but heavily optimized for contiguous memory accesses.